Purpose

ORC-0 — orchestration/ Purpose + boundaries

Piece A — Purpose (what orchestration/ is)

File: `Axion/libraries/orchestration/ORC-0_purpose.md`

---
library: orchestration
id: ORC-0
schema_version: 1.0.0
status: draft

ORC-0 — orchestration/ Purpose + Boundaries

Purpose
`orchestration/` defines the **pipeline execution contract**: the ordered stages, stage IO expectations, run lifecycle semantics (start/resume/rerun), and the canonical run-level records that make execution traceable and reproducible.

This library is the source-of-truth for:
- what stages exist
- what order they run in
- what each stage consumes and produces
- how stage results are reported
- how a run is represented end-to-end (manifest)

What it governs (in scope)
- **Pipeline definition**: stage list, stage order, activation rules
- **Stage IO contracts**: inputs, outputs, required artifacts, pointers
- **Stage report format**: what every stage must emit (evidence, errors, produced artifacts)
- **Run manifest format**: single record tracking an entire run across stages
- **Run lifecycle semantics**:
  - queued → running → gated → failed → released/closed
  - resume rules
  - partial runs
  - stage reruns and replay rules

What it does NOT govern (out of scope)
- Gate predicate language / evaluation mechanics → `gates/`
- Risk classes and override policies → `policy/`
- Workspace/projects/pins/adapters/quotas → `system/`

- Intake form schemas and normalization → `intake/`
- Canonical spec schema and unknown handling → `canonical/`
- Standards packs and resolution logic → `standards/`
- Template registry + rendering rules → `templates/`
- Knowledge library selection rules → `knowledge/`
- Planning artifacts (WBS, acceptance maps) → `planning/`
- Verification proof ledger and completion criteria → `verification/`
- Kit packaging rules → `kit/`
- Telemetry event schemas / sinks → `telemetry/`
- Ops standards (SLOs, alert rules) → `ops/`
- Audit ledger definitions → `audit/`

Consumers (what reads orchestration/)
- Control plane runner (state machine driving stages)
- Stage executors (to know IO contract expectations)
- Gate evaluator (to know what evidence/artifacts should exist at each gate point)
- Operator UI (run timeline, stage status, rerun controls)

Determinism requirements
- Stage ordering is stable and versioned.
- Stage IO contracts are explicit and machine-checkable.
- Run manifest is append-only for stage events (no silent edits).
- Rerun rules are deterministic (inputs pinned, outputs re-derived).

Outputs (what orchestration/ produces)
- pipeline definition record(s)
- stage IO contract definitions
- stage report schema and run manifest schema used during execution

Piece B — Boundary checklist (to prevent drift)

File: `Axion/libraries/orchestration/ORC-0_boundary_checklist.md`

---
library: orchestration
id: ORC-0a
schema_version: 1.0.0
status: draft

ORC-0a — Boundary Checklist

A change belongs in `orchestration/` if it answers:
- “What stages exist and in what order do they run?”

- “What does stage Sx consume and produce?”
- “How do we represent a run and stage reports?”
- “How do resume/partial rerun semantics work deterministically?”

A change does NOT belong in `orchestration/` if it answers:
- “How do we evaluate a predicate / gate?” (gates)
- “What risk class rules exist?” (policy)
- “Who can run this / what is pinned / what quotas apply?” (system)
- “What template/knowledge/standards content exists?” (those libraries)

Pipeline definition model

ORC-1 — Pipeline definition model (stages + order)

Piece A — Pipeline model (concept)

File: `Axion/libraries/orchestration/ORC-1_pipeline_definition_model.md`

---
library: orchestration
id: ORC-1
schema_version: 1.0.0
status: draft

ORC-1 — Pipeline Definition Model

Purpose
Define the authoritative model for a pipeline: its stages, ordering, activation rules, and gating points.

Core entities
- **Pipeline**: named definition of stages + order + version
- **Stage**: step in the pipeline with a stable stage_id
- **Activation rule**: conditions that enable/skip a stage (deterministic)
- **Gating point**: stage boundary where a gate must pass before continuing

Required fields
Pipeline must include:
- pipeline_id (stable)
- version
- stage_order (ordered list of stage_id)
- stages map (stage_id → stage definition)
- gate_points (optional list of gates and where they apply)
- created_at, updated_at, owner

Stage definition must include:
- stage_id (stable)
- name
- consumes (input contract refs)
- produces (output contract refs)
- can_rerun (boolean)
- failure_policy (hard_stop vs retryable vs gate_pause)

Piece B — Pipeline definition schema (JSON Schema)

File:
`Axion/libraries/orchestration/schemas/pipeline_definition.v1.schema.json`

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "schema_version": "1.0.0",
  "id": "pipeline_definition.v1",
  "type": "object",
  "required": [
    "pipeline_id",
    "version",
    "stage_order",
    "stages",
    "created_at",
    "updated_at",
    "owner"
  ],
  "properties": {
    "pipeline_id": { "type": "string", "pattern": "^PIPE-[A-Z0-9_]+$" },
    "version": { "type": "string", "minLength": 1 },

    "stage_order": {
      "type": "array",
      "minItems": 1,
      "items": { "type": "string", "pattern": "^S\\d{1,2}_[A-Z0-9_]+$" }
    },

    "stages": {
      "type": "object",
      "additionalProperties": { "$ref": "#/$defs/stage_def" }
    },

    "gate_points": {
      "type": "array",
      "items": { "$ref": "#/$defs/gate_point" }
    },

    "created_at": { "type": "string", "format": "date-time" },
    "updated_at": { "type": "string", "format": "date-time" },
```

```
  "owner": { "type": "string", "minLength": 2 }
},

"$defs": {
  "stage_def": {
    "type": "object",
    "required": ["stage_id", "name", "consumes", "produces", "can_rerun", "failure_policy"],
    "properties": {
      "stage_id": { "type": "string", "pattern": "^S\\d{1,2}_[A-Z0-9_]+$" },
      "name": { "type": "string", "minLength": 2 },

      "consumes": {
        "type": "array",
        "items": { "type": "string", "minLength": 3 },
        "description": "References to stage IO contract IDs"
      },

      "produces": {
        "type": "array",
        "items": { "type": "string", "minLength": 3 },
        "description": "References to artifacts/contracts produced"
      },

      "activation": {
        "type": "object",
        "description": "Deterministic stage activation conditions",
        "properties": {
          "mode": { "type": "string", "enum": ["always", "conditional", "manual"] },
          "requires": {
            "type": "array",
            "items": { "type": "string" },
            "description": "Named conditions evaluated from run context"
          }
        },
        "additionalProperties": false
      },

      "can_rerun": { "type": "boolean" },

      "failure_policy": {
        "type": "string",
        "enum": ["hard_stop", "retryable", "gate_pause"]
      }
    },
```

```
      "additionalProperties": false
    },

    "gate_point": {
      "type": "object",
      "required": ["after_stage", "gate_id"],
      "properties": {
        "after_stage": { "type": "string", "pattern": "^S\\d{1,2}_[A-Z0-9_]+$" },
        "gate_id": { "type": "string", "minLength": 3 },
        "severity": { "type": "string", "enum": ["hard_stop", "pause", "warn"] }
      },
      "additionalProperties": false
    }
  },

  "additionalProperties": false
}
```

Piece C — Determinism rules (ordering + identity)

File: `Axion/libraries/orchestration/ORC-1_determinism_rules.md`

```
---
library: orchestration
id: ORC-1a
schema_version: 1.0.0
status: draft
---

# ORC-1a — Determinism Rules (Pipeline)

- stage_order is authoritative and must match the keys in stages.
- stage_id values are immutable across versions (do not rename; deprecate by versioning).
- Gate points are evaluated in file order after their after_stage boundary.
- Activation rules must be computable from run context only (no hidden state).
- Any pipeline change requires a version bump.
```

Piece D — Starter pipeline definition (matches your Axion stage order)

File: `Axion/libraries/orchestration/registries/pipeline_definition.axion.v1.json`


```
{
  "schema_version": "1.0.0",
  "pipeline_id": "PIPE-AXION",
  "version": "1.0.0",
  "stage_order": [
    "S0_INIT",
    "S1_INGEST_NORMALIZE",
    "S2_INTAKE_VALIDATION",
    "S3_STANDARDS_RESOLUTION",
    "S4_CANONICAL_BUILD",
    "S5_TEMPLATE_SELECTION",
    "S6_PLAN_GENERATION",
    "S7_TEMPLATE_FILL",
    "S8_GATE_EVALUATION",
    "S9_KIT_PACKAGE",
    "S10_CLOSE"
  ],
  "stages": {
    "S0_INIT": {
      "stage_id": "S0_INIT",
      "name": "Init",
      "consumes": [],
      "produces": ["RUN_MANIFEST_INIT"],
      "activation": { "mode": "always", "requires": [] },
      "can_rerun": true,
      "failure_policy": "hard_stop"
    },
    "S1_INGEST_NORMALIZE": {
      "stage_id": "S1_INGEST_NORMALIZE",
      "name": "Ingest + Normalize",
      "consumes": ["INTAKE_SUBMISSION"],
      "produces": ["NORMALIZED_INPUT"],
      "activation": { "mode": "always", "requires": [] },
      "can_rerun": true,
      "failure_policy": "hard_stop"
    },
    "S2_INTAKE_VALIDATION": {
      "stage_id": "S2_INTAKE_VALIDATION",
      "name": "Intake Validation",
      "consumes": ["NORMALIZED_INPUT"],
```

"produces": ["INTAKE_VALIDATION_REPORT"],
"activation": { "mode": "always", "requires": [] },
"can_rerun": true,
"failure_policy": "gate_pause"
},
"S3_STANDARDS_RESOLUTION": {
"stage_id": "S3_STANDARDS_RESOLUTION",
"name": "Standards Resolution",
"consumes": ["NORMALIZED_INPUT"],
"produces": ["STANDARDS_SNAPSHOT"],
"activation": { "mode": "always", "requires": [] },
"can_rerun": true,
"failure_policy": "hard_stop"
},
"S4_CANONICAL_BUILD": {
"stage_id": "S4_CANONICAL_BUILD",
"name": "Canonical Build",
"consumes": ["NORMALIZED_INPUT", "STANDARDS_SNAPSHOT"],
"produces": ["CANONICAL_SPEC", "UNKNOWN_ASSUMPTIONS"],
"activation": { "mode": "always", "requires": [] },
"can_rerun": true,
"failure_policy": "hard_stop"
},
"S5_TEMPLATE_SELECTION": {
"stage_id": "S5_TEMPLATE_SELECTION",
"name": "Template Selection",
"consumes": ["CANONICAL_SPEC", "STANDARDS_SNAPSHOT"],
"produces": ["TEMPLATE_SELECTION"],
"activation": { "mode": "always", "requires": [] },
"can_rerun": true,
"failure_policy": "hard_stop"
},
"S6_PLAN_GENERATION": {
"stage_id": "S6_PLAN_GENERATION",
"name": "Plan Generation",
"consumes": ["CANONICAL_SPEC", "TEMPLATE_SELECTION",
"STANDARDS_SNAPSHOT"],
"produces": ["BUILD_PLAN", "ACCEPTANCE_MAP", "WORK_BREAKDOWN"],
"activation": { "mode": "always", "requires": [] },
"can_rerun": true,
"failure_policy": "hard_stop"
},
"S7_TEMPLATE_FILL": {
"stage_id": "S7_TEMPLATE_FILL",

"name": "Template Fill",
"consumes": ["BUILD_PLAN", "TEMPLATE_SELECTION", "CANONICAL_SPEC", "STANDARDS_SNAPSHOT"],
"produces": ["RENDERED_TEMPLATES", "RENDER_ENVELOPES"],
"activation": { "mode": "always", "requires": [] },
"can_rerun": true,
"failure_policy": "hard_stop"
},
"S8_GATE_EVALUATION": {
"stage_id": "S8_GATE_EVALUATION",
"name": "Gate Evaluation",
"consumes": ["RENDER_ENVELOPES", "BUILD_PLAN", "CANONICAL_SPEC"],
"produces": ["GATE_REPORT", "PROOF_LEDGER"],
"activation": { "mode": "always", "requires": [] },
"can_rerun": true,
"failure_policy": "gate_pause"
},
"S9_KIT_PACKAGE": {
"stage_id": "S9_KIT_PACKAGE",
"name": "Kit Package",
"consumes": ["RENDERED_TEMPLATES", "GATE_REPORT", "PROOF_LEDGER"],
"produces": ["KIT_BUNDLE", "KIT_MANIFEST"],
"activation": { "mode": "always", "requires": [] },
"can_rerun": true,
"failure_policy": "hard_stop"
},
"S10_CLOSE": {
"stage_id": "S10_CLOSE",
"name": "Close",
"consumes": ["KIT_MANIFEST"],
"produces": ["RUN_CLOSED"],
"activation": { "mode": "always", "requires": [] },
"can_rerun": false,
"failure_policy": "hard_stop"
}
},
"gate_points": [
{ "after_stage": "S2_INTAKE_VALIDATION", "gate_id": "G1_INTAKE_VALIDITY", "severity": "pause" },
{ "after_stage": "S4_CANONICAL_BUILD", "gate_id": "G2_CANONICAL_INTEGRITY", "severity": "hard_stop" },
{ "after_stage": "S3_STANDARDS_RESOLUTION", "gate_id": "G3_STANDARDS_RESOLVED", "severity": "hard_stop" },

```
    { "after_stage": "S5_TEMPLATE_SELECTION", "gate_id": "G4_TEMPLATE_SELECTION",
"severity": "hard_stop" },
    { "after_stage": "S7_TEMPLATE_FILL", "gate_id": "G5_TEMPLATE_COMPLETENESS",
"severity": "hard_stop" },
    { "after_stage": "S6_PLAN_GENERATION", "gate_id": "G6_PLAN_COVERAGE", "severity":
"hard_stop" },
    { "after_stage": "S8_GATE_EVALUATION", "gate_id": "G7_VERIFICATION", "severity":
"pause" }
  ],
  "created_at": "2026-03-05T00:00:00Z",
  "updated_at": "2026-03-05T00:00:00Z",
  "owner": "axion"
}
```

Piece E — Validation checklist

File: `Axion/libraries/orchestration/ORC-1_validation_checklist.md`


```
---
library: orchestration
id: ORC-1b
schema_version: 1.0.0
status: draft
---
```


ORC-1b — Validation Checklist

- [ ] pipeline definition matches pipeline_definition.v1 schema
- [ ] stage_order entries all exist as keys in stages
- [ ] each stages[stage_id].stage_id matches its key
- [ ] gate_points.after_stage exists in stage_order
- [ ] pipeline version bumps when stage list/order/contracts change

Stage IO contracts

ORC-2 — Stage IO contracts (consumes / produces)

Piece A — Stage IO contract model (concept)

File: `Axion/libraries/orchestration/ORC-2_stage_io_contracts.md`

```
---
library: orchestration
id: ORC-2
schema_version: 1.0.0
status: draft
---

# ORC-2 — Stage IO Contracts

## Purpose
Make every stage machine-checkable by defining:
- what it consumes (inputs)
- what it must produce (outputs/artifacts)
- what evidence it must report
- what “required for next stage” means

## Definitions
- **IO contract**: a named contract that defines a stage input or output artifact format.
- **Consumes**: references to IO contract IDs required to execute the stage.
- **Produces**: references to artifacts/contract IDs the stage must emit on success.

## Principles
- Contracts are referenced by ID (not by path).
- Concrete file paths are resolved via run manifest and artifact registry.
- A stage can only proceed if its required inputs exist and validate.
```

Piece B — IO contract schema (single contract record)

File: `Axion/libraries/orchestration/schemas/stage_io_contract.v1.schema.json`

```
{
 "$schema": "https://json-schema.org/draft/2020-12/schema",
 "schema_version": "1.0.0",
 "id": "stage_io_contract.v1",
 "type": "object",
```

```
 "required": ["contract_id", "kind", "description", "schema_ref", "required_fields"],
 "properties": {
   "contract_id": { "type": "string", "pattern": "^[A-Z0-9_\\-]+$" },

   "kind": {
     "type": "string",
     "enum": ["input", "output", "report", "artifact"]
   },

   "description": { "type": "string", "minLength": 5 },

   "schema_ref": {
     "type": "object",
     "required": ["schema_id", "schema_version"],
     "properties": {
       "schema_id": { "type": "string", "minLength": 3 },
       "schema_version": { "type": "string", "minLength": 1 }
     },
     "additionalProperties": false
   },

   "required_fields": {
     "type": "array",
     "minItems": 0,
     "items": { "type": "string", "minLength": 1 }
   },

   "validation": {
     "type": "object",
     "properties": {
       "mode": { "type": "string", "enum": ["schema_only", "schema_plus_rules"] },
       "rules_ref": { "type": "string" }
     },
     "additionalProperties": false
   }
 },
 "additionalProperties": false
}
```

Piece C — IO contracts registry schema

File:
`Axion/libraries/orchestration/schemas/stage_io_registry.v1.schema.json`

```
{
 "$schema": "https://json-schema.org/draft/2020-12/schema",
 "schema_version": "1.0.0",
 "id": "stage_io_registry.v1",
 "type": "object",
 "required": ["registry_id", "schema_version", "contracts"],
 "properties": {
   "registry_id": { "type": "string", "minLength": 3 },
   "schema_version": { "type": "string", "minLength": 1 },
   "contracts": {
     "type": "array",
     "minItems": 1,
     "items": { "$ref": "stage_io_contract.v1.schema.json" }
   }
 },
 "additionalProperties": false
}
```

Piece D — Starter IO registry (IDs referenced by your pipeline definition)

File: `Axion/libraries/orchestration/registries/stage_io_registry.axion.v1.json`

```
{
 "schema_version": "1.0.0",
 "registry_id": "stage_io_registry.axion.v1",
 "contracts": [
   {
     "contract_id": "INTAKE_SUBMISSION",
     "kind": "input",
     "description": "Raw intake submission payload (wizard output).",
     "schema_ref": { "schema_id": "intake_submission.v1", "schema_version": "1.0.0" },
     "required_fields": ["submission_id", "answers"]
   },
   {
     "contract_id": "NORMALIZED_INPUT",
     "kind": "artifact",
     "description": "Normalized intake inputs used deterministically downstream.",
     "schema_ref": { "schema_id": "normalized_input.v1", "schema_version": "1.0.0" },
     "required_fields": ["run_profile", "risk_class", "project_name"]
   },
   {
```

```
  "contract_id": "INTAKE_VALIDATION_REPORT",
  "kind": "report",
  "description": "Report of intake validation results and missing fields.",
  "schema_ref": { "schema_id": "intake_validation_report.v1", "schema_version": "1.0.0" },
  "required_fields": ["status", "errors"]
},
{
  "contract_id": "STANDARDS_SNAPSHOT",
  "kind": "artifact",
  "description": "Resolved standards snapshot for the run.",
  "schema_ref": { "schema_id": "standards_snapshot.v1", "schema_version": "1.0.0" },
  "required_fields": ["snapshot_id", "packs", "resolver_version"]
},
{
  "contract_id": "CANONICAL_SPEC",
  "kind": "artifact",
  "description": "Canonical spec built from normalized input + standards.",
  "schema_ref": { "schema_id": "canonical_spec.v1", "schema_version": "1.0.0" },
  "required_fields": ["entities", "relationships"]
},
{
  "contract_id": "UNKNOWN_ASSUMPTIONS",
  "kind": "artifact",
  "description": "Unknowns/assumptions list produced during canonical build.",
  "schema_ref": { "schema_id": "unknown_assumptions.v1", "schema_version": "1.0.0" },
  "required_fields": ["unknowns"]
},
{
  "contract_id": "TEMPLATE_SELECTION",
  "kind": "artifact",
  "description": "Selected templates (ordered) with rationale and constraints.",
  "schema_ref": { "schema_id": "template_selection.v1", "schema_version": "1.0.0" },
  "required_fields": ["templates", "selection_reason"]
},
{
  "contract_id": "WORK_BREAKDOWN",
  "kind": "artifact",
  "description": "Work breakdown structure derived from canonical spec and template set.",
  "schema_ref": { "schema_id": "work_breakdown.v1", "schema_version": "1.0.0" },
  "required_fields": ["items"]
},
{
  "contract_id": "ACCEPTANCE_MAP",
  "kind": "artifact",
```

"description": "Acceptance criteria mapping (requirements → artifacts/tests).",
"schema_ref": { "schema_id": "acceptance_map.v1", "schema_version": "1.0.0" },
"required_fields": ["requirements"]
},
{
"contract_id": "BUILD_PLAN",
"kind": "artifact",
"description": "Build plan and sequencing decisions for the run.",
"schema_ref": { "schema_id": "build_plan.v1", "schema_version": "1.0.0" },
"required_fields": ["sequence"]
},
{
"contract_id": "RENDERED_TEMPLATES",
"kind": "artifact",
"description": "The rendered docs output from template fill (content files).",
"schema_ref": { "schema_id": "rendered_templates.v1", "schema_version": "1.0.0" },
"required_fields": ["outputs"]
},
{
"contract_id": "RENDER_ENVELOPES",
"kind": "artifact",
"description": "Render envelopes including citations and trace metadata.",
"schema_ref": { "schema_id": "render_envelopes.v1", "schema_version": "1.0.0" },
"required_fields": ["envelopes"]
},
{
"contract_id": "GATE_REPORT",
"kind": "report",
"description": "Gate evaluation report including failures and evidence pointers.",
"schema_ref": { "schema_id": "gate_report.v1", "schema_version": "1.0.0" },
"required_fields": ["gate_id", "status", "evidence"]
},
{
"contract_id": "PROOF_LEDGER",
"kind": "artifact",
"description": "Proof ledger containing verification records for completion.",
"schema_ref": { "schema_id": "proof_ledger.v1", "schema_version": "1.0.0" },
"required_fields": ["proofs"]
},
{
"contract_id": "KIT_MANIFEST",
"kind": "artifact",
"description": "Kit manifest of packaged deliverables.",
"schema_ref": { "schema_id": "kit_manifest.v1", "schema_version": "1.0.0" },

```
    "required_fields": ["kit_id", "contents"]
  }
 ]
}
```

Piece E — Determinism rules (IO contract enforcement)

File: `Axion/libraries/orchestration/ORC-2_determinism_rules.md`

```
---
library: orchestration
id: ORC-2a
schema_version: 1.0.0
status: draft
---

# ORC-2a — Determinism Rules (Stage IO)

- Stages may only consume/produce contract IDs defined in the IO registry.
- Contract IDs are stable; contract meaning changes require schema version bump.
- Input validation is performed before stage execution begins.
- A stage success must include all required produced contracts listed in pipeline definition.
- Any missing required produced artifact is a stage failure.
```

Piece F — Validation checklist

File: `Axion/libraries/orchestration/ORC-2_validation_checklist.md`

```
---
library: orchestration
id: ORC-2b
schema_version: 1.0.0
status: draft
---

# ORC-2b — Validation Checklist

- [ ] stage_io_registry validates against stage_io_registry schema
- [ ] every contract has schema_ref (schema_id + schema_version)
- [ ] required_fields are defined (even if empty)
- [ ] pipeline_definition consumes/produces references exist in stage_io_registry
- [ ] stage execution validates consumes before running
```

- [ ] stage completion verifies all produces exist and validate

Run manifest format

ORC-3 — Run manifest format (tracks an entire pipeline execution)

Piece A — Run manifest model (concept)

File: `Axion/libraries/orchestration/ORC-3_run_manifest_model.md`

```
---
library: orchestration
id: ORC-3
schema_version: 1.0.0
status: draft
---

# ORC-3 — Run Manifest Model

## Purpose
The run manifest is the single, authoritative record of a pipeline run:
- which pipeline version ran
- which pins/locks were resolved
- which stages executed (and their outcomes)
- what artifacts were produced and where they live
- which gates passed/failed and why
- how/when the run was resumed or rerun

## Key properties
- Append-only event log semantics (no silent edits)
- Deterministic pointers to artifacts (path + hash optional)
- Stable stage timeline entries (start/end/status)
- Captures policy/quota/adapters decisions from `system/`

## Consumers
- Orchestrator (resume/rerun)
- Operator UI (timeline)
- Gate evaluator (evidence pointers)
- Kit packager (collect outputs)
```

Piece B — Run manifest schema (JSON Schema)

File: `Axion/libraries/orchestration/schemas/run_manifest.v1.schema.json`

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "schema_version": "1.0.0",
  "id": "run_manifest.v1",
  "type": "object",
  "required": [
    "run_id",
    "created_at",
    "updated_at",
    "workspace_id",
    "project_id",
    "pipeline_ref",
    "pins",
    "runtime",
    "stage_timeline",
    "artifacts"
  ],
  "properties": {
    "run_id": { "type": "string", "pattern": "^RUN-[A-Z0-9]{6,}$" },

    "created_at": { "type": "string", "format": "date-time" },
    "updated_at": { "type": "string", "format": "date-time" },

    "workspace_id": { "type": "string", "pattern": "^WS-[A-Z0-9]{6,}$" },
    "project_id": { "type": "string", "pattern": "^PRJ-[A-Z0-9]{6,}$" },

    "pipeline_ref": {
      "type": "object",
      "required": ["pipeline_id", "version", "path"],
      "properties": {
        "pipeline_id": { "type": "string" },
        "version": { "type": "string" },
        "path": { "type": "string" },
        "hash": { "type": "string" }
      },
      "additionalProperties": false
    },

    "pins": {
      "type": "object",
      "required": ["pin_set_id", "pin_set_ref"],
      "properties": {
        "pin_set_id": { "type": "string" },
        "pin_set_ref": {
```

```
      "type": "object",
      "required": ["version", "path"],
      "properties": {
        "version": { "type": "string" },
        "path": { "type": "string" },
        "hash": { "type": "string" }
      },
      "additionalProperties": false
    }
  },
  "additionalProperties": false
},

"runtime": {
  "type": "object",
  "required": ["profile_id", "risk_class", "adapter_profile_id", "quota_effective"],
  "properties": {
    "profile_id": { "type": "string" },
    "risk_class": { "type": "string" },

    "adapter_profile_id": { "type": "string" },

    "quota_effective": {
      "type": "object",
      "description": "Snapshot of effective quota decision at run start",
      "additionalProperties": true
    },

    "policy_decisions": {
      "type": "array",
      "items": {
        "type": "object",
        "required": ["decision_id", "hook_point", "outcome", "expires_at"],
        "properties": {
          "decision_id": { "type": "string" },
          "hook_point": { "type": "string" },
          "outcome": { "type": "string" },
          "expires_at": { "type": "string", "format": "date-time" }
        },
        "additionalProperties": true
      }
    }
  },
  "additionalProperties": false
```

```
    },

    "stage_timeline": {
      "type": "array",
      "minItems": 1,
      "items": { "$ref": "#/$defs/stage_entry" }
    },

    "gates": {
      "type": "array",
      "items": { "$ref": "#/$defs/gate_entry" }
    },

    "artifacts": {
      "type": "array",
      "items": { "$ref": "#/$defs/artifact_entry" }
    },

    "events": {
      "type": "array",
      "description": "Append-only run events (resume, rerun, approvals, notifications)",
      "items": { "$ref": "#/$defs/run_event" }
    }
  },

  "$defs": {
    "stage_entry": {
      "type": "object",
      "required": ["stage_id", "status", "started_at"],
      "properties": {
        "stage_id": { "type": "string" },
        "status": { "type": "string", "enum": ["queued", "running", "succeeded", "failed", "paused",
"skipped"] },
        "started_at": { "type": "string", "format": "date-time" },
        "ended_at": { "type": "string", "format": "date-time" },
        "report_ref": {
          "type": "object",
          "properties": { "path": { "type": "string" }, "hash": { "type": "string" } },
          "additionalProperties": false
        },
        "notes": { "type": "array", "items": { "type": "string" } }
      },
      "additionalProperties": false
    },
```

```
  "gate_entry": {
    "type": "object",
    "required": ["gate_id", "status", "evaluated_at"],
    "properties": {
      "gate_id": { "type": "string" },
      "status": { "type": "string", "enum": ["pass", "fail", "warn"] },
      "evaluated_at": { "type": "string", "format": "date-time" },
      "gate_report_ref": {
        "type": "object",
        "properties": { "path": { "type": "string" }, "hash": { "type": "string" } },
        "additionalProperties": false
      }
    },
    "additionalProperties": false
  },

  "artifact_entry": {
    "type": "object",
    "required": ["artifact_id", "contract_id", "path", "produced_by_stage"],
    "properties": {
      "artifact_id": { "type": "string", "minLength": 2 },
      "contract_id": { "type": "string", "minLength": 2 },
      "path": { "type": "string", "minLength": 2 },
      "hash": { "type": "string" },
      "produced_by_stage": { "type": "string" }
    },
    "additionalProperties": false
  },

  "run_event": {
    "type": "object",
    "required": ["event_type", "at"],
    "properties": {
      "event_type": {
        "type": "string",
        "enum": ["RUN_STARTED", "RUN_RESUMED", "STAGE_RERUN",
"APPROVAL_GRANTED", "NOTIFICATION_SENT"]
      },
      "at": { "type": "string", "format": "date-time" },
      "data": { "type": "object", "additionalProperties": true }
    },
    "additionalProperties": false
  }
```

```
  },

  "additionalProperties": false
}
```

Piece C — Manifest invariants (append-only + stable pointers)

File: `Axion/libraries/orchestration/ORC-3_invariants.md`

```
---
library: orchestration
id: ORC-3a
schema_version: 1.0.0
status: draft
---

# ORC-3a — Run Manifest Invariants

- The manifest must record the pipeline_ref (pipeline_id + version + path).
- The manifest must record resolved pin_set_ref before stage execution.
- Stage timeline entries are append-only:
  - status transitions must be monotonic (queued → running →
succeeded/failed/paused/skipped).
- Artifact entries must include:
  - contract_id (from stage IO registry)
  - produced_by_stage (stage_id)
  - path (deterministic location)
- Reruns/resumes must emit a run_event with supporting data.
```

Piece D — Validation checklist

File: `Axion/libraries/orchestration/ORC-3_validation_checklist.md`

```
---
library: orchestration
id: ORC-3b
schema_version: 1.0.0
status: draft
---
```

ORC-3b — Validation Checklist

- [ ] run_manifest validates against run_manifest.v1 schema
- [ ] pipeline_ref is present and points to a pinned pipeline definition
- [ ] pins.pin_set_ref present before any stage status == running
- [ ] stage_timeline status transitions are valid
- [ ] artifacts.contract_id exists in stage IO registry (strict mode)
- [ ] artifacts.produced_by_stage exists in stage_order (strict mode)
- [ ] rerun/resume actions are recorded in events

Stage report schema

ORC-4 — Stage report schema (standard report every stage emits)

Piece A — Stage report model (concept)

File: `Axion/libraries/orchestration/ORC-4_stage_report_model.md`

```
---
library: orchestration
id: ORC-4
schema_version: 1.0.0
status: draft
---

# ORC-4 — Stage Report Model

## Purpose
Every stage emits a stage report so operators and downstream gates can understand:
- what inputs were used (refs)
- what outputs were produced (refs)
- what validations ran
- what errors/warnings occurred
- what evidence is available for gates

## Principles
- Reports are machine-readable and uniform across stages.
- Reports point to artifacts by path + optional hash (not embedded content).
- Reports capture the “why” minimally (decisions, not essays).
```

Piece B — Stage report schema (JSON Schema)

File: `Axion/libraries/orchestration/schemas/stage_report.v1.schema.json`

```
{
 "$schema": "https://json-schema.org/draft/2020-12/schema",
 "schema_version": "1.0.0",
 "id": "stage_report.v1",
 "type": "object",
 "required": [
   "run_id",
   "stage_id",
```

```
  "status",
  "started_at",
  "ended_at",
  "inputs",
  "outputs",
  "validations",
  "messages"
],
"properties": {
  "run_id": { "type": "string", "pattern": "^RUN-[A-Z0-9]{6,}$" },
  "stage_id": { "type": "string" },

  "status": { "type": "string", "enum": ["succeeded", "failed", "paused", "skipped"] },

  "started_at": { "type": "string", "format": "date-time" },
  "ended_at": { "type": "string", "format": "date-time" },

  "inputs": {
    "type": "array",
    "items": { "$ref": "#/$defs/io_ref" }
  },

  "outputs": {
    "type": "array",
    "items": { "$ref": "#/$defs/io_ref" }
  },

  "validations": {
    "type": "array",
    "description": "Schema and rule validations performed by the stage",
    "items": {
      "type": "object",
      "required": ["validation_id", "status"],
      "properties": {
        "validation_id": { "type": "string", "minLength": 2 },
        "status": { "type": "string", "enum": ["pass", "fail", "warn"] },
        "details": { "type": "object", "additionalProperties": true }
      },
      "additionalProperties": false
    }
  },

  "messages": {
    "type": "array",
```

```
    "description": "Human-readable warnings/errors (short)",
    "items": {
      "type": "object",
      "required": ["level", "code", "text"],
      "properties": {
        "level": { "type": "string", "enum": ["info", "warning", "error"] },
        "code": { "type": "string", "minLength": 2 },
        "text": { "type": "string", "minLength": 2 }
      },
      "additionalProperties": false
    }
  },

  "decisions": {
    "type": "array",
    "description": "Optional: key decisions made (selector choices, resolver outputs, etc.)",
    "items": {
      "type": "object",
      "required": ["decision_id", "summary"],
      "properties": {
        "decision_id": { "type": "string", "minLength": 2 },
        "summary": { "type": "string", "minLength": 3 },
        "refs": { "type": "array", "items": { "type": "string" } }
      },
      "additionalProperties": false
    }
  }
},

"$defs": {
  "io_ref": {
    "type": "object",
    "required": ["contract_id", "path"],
    "properties": {
      "contract_id": { "type": "string", "minLength": 2 },
      "path": { "type": "string", "minLength": 2 },
      "hash": { "type": "string" }
    },
    "additionalProperties": false
  }
},

"additionalProperties": false
}
```

Piece C — Determinism rules (report integrity)

File: `Axion/libraries/orchestration/ORC-4_determinism_rules.md`

```
---
library: orchestration
id: ORC-4a
schema_version: 1.0.0
status: draft
---

# ORC-4a — Determinism Rules (Stage Reports)

- Every executed stage must emit exactly one stage_report.v1.
- inputs/outputs must reference contract IDs from the stage IO registry.
- paths must match the artifact entries recorded in the run manifest (strict mode).
- messages are short and stable (no large freeform dumps).
- decisions (if present) must reference deterministic IDs (template ids, bundle ids, snapshot ids).
```

Piece D — Minimal example stage report (template)

File: `Axion/libraries/orchestration/templates/stage_report.example.json`

```
{
 "run_id": "RUN-ABC123",
 "stage_id": "S3_STANDARDS_RESOLUTION",
 "status": "succeeded",
 "started_at": "2026-03-05T00:00:00Z",
 "ended_at": "2026-03-05T00:01:00Z",
 "inputs": [
   { "contract_id": "NORMALIZED_INPUT", "path":
"runs/RUN-ABC123/artifacts/NORMALIZED_INPUT.json" }
 ],
 "outputs": [
   { "contract_id": "STANDARDS_SNAPSHOT", "path":
"runs/RUN-ABC123/artifacts/STANDARDS_SNAPSHOT.json" }
 ],
 "validations": [
   { "validation_id": "STD-SNAPSHOT-SCHEMA", "status": "pass", "details": {} }
 ],
 "messages": [
```

{ "level": "info", "code": "STD-RESOLVED", "text": "Resolved 5 standards packs." }
],
"decisions": [
{ "decision_id": "STD-RESOLVE-001", "summary": "Applied resolver order: global -> profile -> risk.", "refs": ["snapshot:STDSNAP-001"] }
]
}

Piece E — Validation checklist

File: `Axion/libraries/orchestration/ORC-4_validation_checklist.md`

```
---
library: orchestration
id: ORC-4b
schema_version: 1.0.0
status: draft
---

# ORC-4b — Validation Checklist

- [ ] stage_report validates against stage_report.v1 schema
- [ ] inputs/outputs contract_id exist in stage IO registry (strict mode)
- [ ] stage_id exists in pipeline stage_order (strict mode)
- [ ] stage_report paths align with run manifest artifact entries (strict mode)
- [ ] exactly one report per executed stage
```

Rerun / resume rules

ORC-5 — Rerun / resume rules (partial runs, stage reruns)

Piece A — Rerun/resume model (concept)

File: `Axion/libraries/orchestration/ORC-5_rerun_resume_model.md`

```
---
library: orchestration
id: ORC-5
schema_version: 1.0.0
status: draft
---

# ORC-5 — Rerun / Resume Model

## Purpose
Define deterministic behavior for:
- resuming a paused/failed run
- rerunning a stage (repair or regeneration)
- partial runs (starting from a stage with pinned inputs)

## Key idea
Reruns are only valid if inputs are pinned and the run manifest records:
- why the rerun occurred
- which stage(s) were rerun
- which artifacts were invalidated/replaced
- what was kept stable (pins, standards snapshot, selection artifacts)

## Definitions
- **Resume**: continue a paused run without changing upstream artifacts/pins.
- **Stage rerun**: re-execute a specific stage using the same pinned upstream inputs.
- **Partial run**: start execution from stage N by providing pinned inputs equivalent to what earlier stages would have produced.
```

Piece B — Allowed rerun operations registry (what can rerun + what gets invalidated)

File:
`Axion/libraries/orchestration/registries/rerun_policies.axion.v1.json`

{
  "schema_version": "1.0.0",
  "registry_id": "rerun_policies.axion.v1",
  "rules": [
    {
      "stage_id": "S2_INTAKE_VALIDATION",
      "can_rerun": true,
      "invalidates": []
    },
    {
      "stage_id": "S3_STANDARDS_RESOLUTION",
      "can_rerun": true,
      "invalidates": ["STANDARDS_SNAPSHOT", "CANONICAL_SPEC", "UNKNOWN_ASSUMPTIONS", "TEMPLATE_SELECTION", "BUILD_PLAN", "WORK_BREAKDOWN", "ACCEPTANCE_MAP", "RENDERED_TEMPLATES", "RENDER_ENVELOPES", "GATE_REPORT", "PROOF_LEDGER", "KIT_MANIFEST"]
    },
    {
      "stage_id": "S4_CANONICAL_BUILD",
      "can_rerun": true,
      "invalidates": ["CANONICAL_SPEC", "UNKNOWN_ASSUMPTIONS", "TEMPLATE_SELECTION", "BUILD_PLAN", "WORK_BREAKDOWN", "ACCEPTANCE_MAP", "RENDERED_TEMPLATES", "RENDER_ENVELOPES", "GATE_REPORT", "PROOF_LEDGER", "KIT_MANIFEST"]
    },
    {
      "stage_id": "S5_TEMPLATE_SELECTION",
      "can_rerun": true,
      "invalidates": ["TEMPLATE_SELECTION", "BUILD_PLAN", "WORK_BREAKDOWN", "ACCEPTANCE_MAP", "RENDERED_TEMPLATES", "RENDER_ENVELOPES", "GATE_REPORT", "PROOF_LEDGER", "KIT_MANIFEST"]
    },
    {
      "stage_id": "S6_PLAN_GENERATION",
      "can_rerun": true,
      "invalidates": ["BUILD_PLAN", "WORK_BREAKDOWN", "ACCEPTANCE_MAP", "RENDERED_TEMPLATES", "RENDER_ENVELOPES", "GATE_REPORT", "PROOF_LEDGER", "KIT_MANIFEST"]
    },
    {
      "stage_id": "S7_TEMPLATE_FILL",
      "can_rerun": true,
      "invalidates": ["RENDERED_TEMPLATES", "RENDER_ENVELOPES", "GATE_REPORT", "PROOF_LEDGER", "KIT_MANIFEST"]

```
    },
    {
      "stage_id": "S8_GATE_EVALUATION",
      "can_rerun": true,
      "invalidates": ["GATE_REPORT", "PROOF_LEDGER", "KIT_MANIFEST"]
    },
    {
      "stage_id": "S9_KIT_PACKAGE",
      "can_rerun": true,
      "invalidates": ["KIT_MANIFEST"]
    }
  ]
}
```

Piece C — Rerun request schema (operator or system-triggered)

File: `Axion/libraries/orchestration/schemas/rerun_request.v1.schema.json`

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "schema_version": "1.0.0",
  "id": "rerun_request.v1",
  "type": "object",
  "required": ["request_id", "run_id", "stage_id", "mode", "reason", "requested_by",
"requested_at"],
  "properties": {
    "request_id": { "type": "string", "pattern": "^RERUN-[A-Z0-9]{6,}$" },
    "run_id": { "type": "string", "pattern": "^RUN-[A-Z0-9]{6,}$" },
    "stage_id": { "type": "string" },

    "mode": {
      "type": "string",
      "enum": ["resume", "stage_rerun", "partial_run"]
    },

    "reason": { "type": "string", "minLength": 3 },
    "requested_by": { "type": "string", "minLength": 2 },
    "requested_at": { "type": "string", "format": "date-time" },

    "pins_override": {
      "type": "object",
      "description": "Only allowed if SYS pin policy allows; otherwise gate failure.",
```

```
      "additionalProperties": true
    },

    "partial_inputs": {
      "type": "array",
      "description": "For partial_run: list of contract_id + path refs that substitute upstream
stages.",
      "items": {
        "type": "object",
        "required": ["contract_id", "path"],
        "properties": {
          "contract_id": { "type": "string" },
          "path": { "type": "string" },
          "hash": { "type": "string" }
        },
        "additionalProperties": false
      }
    }
  },
  "additionalProperties": false
}
```

Piece D — Resume/rerun invariants (what must remain stable)

File: `Axion/libraries/orchestration/ORC-5_invariants.md`

```
---
library: orchestration
id: ORC-5a
schema_version: 1.0.0
status: draft
---

# ORC-5a — Rerun/Resume Invariants

## Resume invariants
Resume is allowed only if:
- pins.pin_set_ref is unchanged
- pipeline_ref is unchanged
- upstream artifacts are unchanged (no invalidation)

## Stage rerun invariants
```

Stage rerun is allowed only if:
- can_rerun == true for the stage
- pins.pin_set_ref is unchanged unless SYS pin policy allows override + policy approves
- all consumes contracts required by the stage exist and validate

Partial run invariants
Partial run is allowed only if:
- partial_inputs provide all contracts needed for the starting stage's consumes chain
- all partial_inputs are pinned refs (path + hash or versioned ref)
- run manifest records this as a new run_id OR as a distinct rerun event with a new pin_set_id (policy choice)

Invalidation behavior
- When rerunning a stage, invalidate downstream artifacts listed in rerun policies.
- Invalidation must be recorded in run events:
  - what contract_ids were invalidated
  - what new artifact paths replaced them

Piece E — Run manifest event requirements (how reruns are recorded)

File: `Axion/libraries/orchestration/ORC-5_manifest_events.md`

---
library: orchestration
id: ORC-5b
schema_version: 1.0.0
status: draft

ORC-5b — Run Manifest Events for Reruns

On rerun/resume, the manifest must append an event:

RUN_RESUMED
- at
- data: { reason, requested_by }

STAGE_RERUN
- at
- data: { stage_id, reason, requested_by, invalidated_contract_ids[], new_outputs[] }

PARTIAL_RUN (optional event_type if you split it)

- at
- data: { start_stage, partial_inputs[], reason, requested_by }

All events must reference stable IDs and paths, not inline content.

Piece F — Validation checklist

File: `Axion/libraries/orchestration/ORC-5_validation_checklist.md`

```
---
library: orchestration
id: ORC-5c
schema_version: 1.0.0
status: draft
---

# ORC-5c — Validation Checklist

- [ ] rerun_policies registry exists and stage_id entries are valid
- [ ] rerun_request validates against schema
- [ ] resume does not change pins or pipeline_ref
- [ ] stage_rerun only allowed when can_rerun==true
- [ ] downstream invalidation list is applied and recorded
- [ ] run manifest appends rerun/resume events with required data
```

Gates

ORC-6 — Gates for orchestration (stage contract compliance)

Piece A — Gate intent (what orchestration must enforce)

File: `Axion/libraries/orchestration/ORC-6_orchestration_gates.md`

```
---
library: orchestration
id: ORC-6
schema_version: 1.0.0
status: draft
---

# ORC-6 — Orchestration Gates (Stage Contract Compliance)

## Purpose
Ensure stages cannot drift from the declared pipeline + IO contracts.

## Gate set (minimum)

### ORC-GATE-01 — Stage order integrity
- Stage execution must follow pipeline.stage_order.
- Skips must be justified by activation rules and recorded.

### ORC-GATE-02 — Consumes exist and validate
- Before a stage executes, all consumes contract artifacts must exist and validate against
schema_ref.

### ORC-GATE-03 — Produces complete
- On stage success, all produces contract artifacts must exist and validate.

### ORC-GATE-04 — Stage report emitted
- Every executed stage must emit exactly one stage_report.v1 with correct input/output refs.

### ORC-GATE-05 — Run manifest coherence
- Run manifest must include:
  - pipeline_ref
  - pins.pin_set_ref
  - stage timeline entries aligned with emitted reports
  - artifact entries aligned with stage outputs
```

ORC-GATE-06 — Rerun/resume invariants
- Resume and rerun actions must comply with ORC-5 invariants and be recorded as events.

Piece B — Machine-readable gate spec (for gate runner)

File: `Axion/libraries/orchestration/ORC-6_orchestration_gates.spec.json`


```
{
  "schema_version": "1.0.0",
  "contract_id": "ORC-6a",
  "gates": [
    {
      "gate_id": "ORC-GATE-01",
      "name": "Stage order integrity",
      "severity": "hard_stop",
      "checks": [
        "executed_stage_ids is subsequence of pipeline.stage_order",
        "any skipped stages have activation justification in manifest"
      ]
    },
    {
      "gate_id": "ORC-GATE-02",
      "name": "Consumes exist and validate",
      "severity": "hard_stop",
      "checks": [
        "before stage starts: all consumes artifacts exist",
        "before stage starts: consumes validate against schema_ref"
      ]
    },
    {
      "gate_id": "ORC-GATE-03",
      "name": "Produces complete",
      "severity": "hard_stop",
      "checks": [
        "if stage status==succeeded then all produces artifacts exist",
        "if stage status==succeeded then produces validate against schema_ref"
      ]
    },
    {
      "gate_id": "ORC-GATE-04",
      "name": "Stage report emitted",
      "severity": "hard_stop",
```

```
      "checks": [
        "exactly one stage_report per executed stage",
        "stage_report.inputs/outputs align with manifest artifact refs"
      ]
    },
    {
      "gate_id": "ORC-GATE-05",
      "name": "Run manifest coherence",
      "severity": "hard_stop",
      "checks": [
        "manifest has pipeline_ref and pins.pin_set_ref",
        "stage timeline aligns with emitted stage reports",
        "artifact entries align with stage outputs"
      ]
    },
    {
      "gate_id": "ORC-GATE-06",
      "name": "Rerun/resume invariants",
      "severity": "hard_stop",
      "checks": [
        "resume does not change pins/pipeline_ref",
        "stage_rerun respects can_rerun and invalidation rules",
        "rerun/resume events appended to manifest"
      ]
    }
  ]
}
```

Piece C — Evidence format (what a failure report must include)

File: `Axion/libraries/orchestration/ORC-6_gate_evidence_format.md`

```
---
library: orchestration
id: ORC-6b
schema_version: 1.0.0
status: draft
---

# ORC-6b — Orchestration Gate Evidence Format

Every ORC gate failure report MUST include:
```

- gate_id
- run_id
- stage_id (if applicable)
- failing_contract_ids (if IO-related)
- expected vs actual (short)
- pointers (paths to manifest, stage report, artifacts)
- remediation guidance (what to fix)

Examples:
- ORC-GATE-02: missing consumes artifact NORMALIZED_INPUT for stage S3
- ORC-GATE-03: produced artifact STANDARDS_SNAPSHOT failed schema validation
- ORC-GATE-01: stage executed out of order

Piece D — Validation checklist

File: `Axion/libraries/orchestration/ORC-6_validation_checklist.md`

---
library: orchestration
id: ORC-6c
schema_version: 1.0.0
status: draft

ORC-6c — Validation Checklist

- [ ] ORC gates are defined with IDs ORC-GATE-01..06
- [ ] Each gate has deterministic checks
- [ ] Evidence format includes expected vs actual + pointers
- [ ] Gates reference pipeline definition + stage IO registry + run manifest + stage reports

Minimum viable orchestration

ORC-7 — Minimum viable orchestration set (runtime-required files + “done”)

Piece A — MV orchestration inventory

File: `Axion/libraries/orchestration/ORC-7_minimum_viable_set.md`

---
library: orchestration
id: ORC-7
schema_version: 1.0.0
status: draft

ORC-7 — Minimum Viable orchestration/ Set

Goal
Define the smallest orchestration library set required to execute runs deterministically.

Required runtime files (minimum)
1) Core docs
- ORC-0_purpose.md
- ORC-0_boundary_checklist.md
- ORC-1_pipeline_definition_model.md
- ORC-1_determinism_rules.md
- ORC-1_validation_checklist.md
- ORC-2_stage_io_contracts.md
- ORC-2_determinism_rules.md
- ORC-2_validation_checklist.md
- ORC-3_run_manifest_model.md
- ORC-3_invariants.md
- ORC-3_validation_checklist.md
- ORC-4_stage_report_model.md
- ORC-4_determinism_rules.md
- ORC-4_validation_checklist.md
- ORC-5_rerun_resume_model.md
- ORC-5_invariants.md
- ORC-5_manifest_events.md
- ORC-5_validation_checklist.md
- ORC-6_orchestration_gates.md
- ORC-6_gate_evidence_format.md
- ORC-6_validation_checklist.md

- ORC-7_minimum_viable_set.md

2) Schemas (must be in schema_registry)
- schemas/pipeline_definition.v1.schema.json
- schemas/stage_io_contract.v1.schema.json
- schemas/stage_io_registry.v1.schema.json
- schemas/run_manifest.v1.schema.json
- schemas/stage_report.v1.schema.json
- schemas/rerun_request.v1.schema.json

3) Registries (runtime defaults)
- registries/pipeline_definition.axion.v1.json
- registries/stage_io_registry.axion.v1.json
- registries/rerun_policies.axion.v1.json

4) Gate spec
- ORC-6_orchestration_gates.spec.json

Piece B — Definition of done (orchestration/)

File: `Axion/libraries/orchestration/ORC-7_definition_of_done.md`

---
library: orchestration
id: ORC-7a
schema_version: 1.0.0
status: draft

ORC-7a — orchestration/ Definition of Done

orchestration/ is “done” when:

Schemas + registries
- [ ] All orchestration schemas validate (JSON Schema check)
- [ ] pipeline_definition registry validates and matches expected stage order
- [ ] stage_io_registry validates and covers all consumes/produces used by pipeline_definition
- [ ] rerun_policies registry validates

Runtime behavior (contract-level)
- [ ] A run produces a valid run_manifest with:
  - pipeline_ref pinned

  - pins.pin_set_ref present before stage execution
  - stage_timeline entries aligned with stage reports
  - artifact entries aligned with stage outputs
- [ ] Each executed stage produces exactly one stage_report
- [ ] Stage consumes are validated before execution (ORC-GATE-02)
- [ ] Stage produces are validated after success (ORC-GATE-03)

Rerun/resume
- [ ] Resume does not change pins/pipeline_ref
- [ ] Stage rerun invalidates downstream artifacts deterministically and records events
- [ ] Partial runs require pinned substitute inputs and record manifest events

Gates
- [ ] ORC-GATE-01..06 are runnable and produce evidence reports on failure

---

Piece C — Minimal folder tree

File: `Axion/libraries/orchestration/ORC-7_minimal_tree.txt`

```
Axion/libraries/orchestration/
  ORC-0_purpose.md
  ORC-0_boundary_checklist.md
  ORC-1_pipeline_definition_model.md
  ORC-1_determinism_rules.md
  ORC-1_validation_checklist.md
  ORC-2_stage_io_contracts.md
  ORC-2_determinism_rules.md
  ORC-2_validation_checklist.md
  ORC-3_run_manifest_model.md
  ORC-3_invariants.md
  ORC-3_validation_checklist.md
  ORC-4_stage_report_model.md
  ORC-4_determinism_rules.md
  ORC-4_validation_checklist.md
  ORC-5_rerun_resume_model.md
  ORC-5_invariants.md
  ORC-5_manifest_events.md
  ORC-5_validation_checklist.md
  ORC-6_orchestration_gates.md
  ORC-6_orchestration_gates.spec.json
  ORC-6_gate_evidence_format.md
  ORC-6_validation_checklist.md
```

```
ORC-7_minimum_viable_set.md
ORC-7_definition_of_done.md
ORC-7_minimal_tree.txt

schemas/
  pipeline_definition.v1.schema.json
  stage_io_contract.v1.schema.json
  stage_io_registry.v1.schema.json
  run_manifest.v1.schema.json
  stage_report.v1.schema.json
  rerun_request.v1.schema.json

registries/
  pipeline_definition.axion.v1.json
  stage_io_registry.axion.v1.json
  rerun_policies.axion.v1.json

templates/
  stage_report.example.json
```